Module 10: Ingress, NetworkPolicy, Helm & the Road Ahead

Networking, policy, packaging — and the 2026 Kubernetes direction

From “it runs” to “it is safely exposed, constrained, packaged, and ready to evolve.”

FINAL MODULE • KUBERNETES TRAINING SERIES

From containers to platform thinking

10

Modules 1–9 built the operational foundation. This finale connects networking, security boundaries, and packaging.

01 Docker	**02 Architecture**	**03 Pods/Services**	**04 Deployments**	**04B Services**
05 Cluster Ops	**06 Config**	**07 Scheduling**	**08 Storage**	**09 RBAC**

Agenda A • Ingress

Expose multiple Services through one HTTP entry point.

Agenda B • NetworkPolicy

Turn an open Pod network into an explicit allow-list.

Agenda C • Helm + Road Ahead

Package apps, then understand the 2026 Gateway API shift.

Series continuity: the final module deliberately reuses Service, Deployment, RBAC and controller-loop mental models.

Four ways to expose a Service

A

A single cluster can use all four patterns — the choice depends on scope and traffic semantics.

ClusterIP

Internal only

Service-to-Service

NodePort

Node address + static port

Direct L4 access

LoadBalancer

Cloud/provider LB

External L4

Ingress

HTTP/HTTPS routing

Many Services • one entry point

2026 nuance: the Ingress API remains stable but frozen; Kubernetes now recommends Gateway API for new long-term designs.

Runbook framing: Ingress is the preferred single entry point for multiple Services.

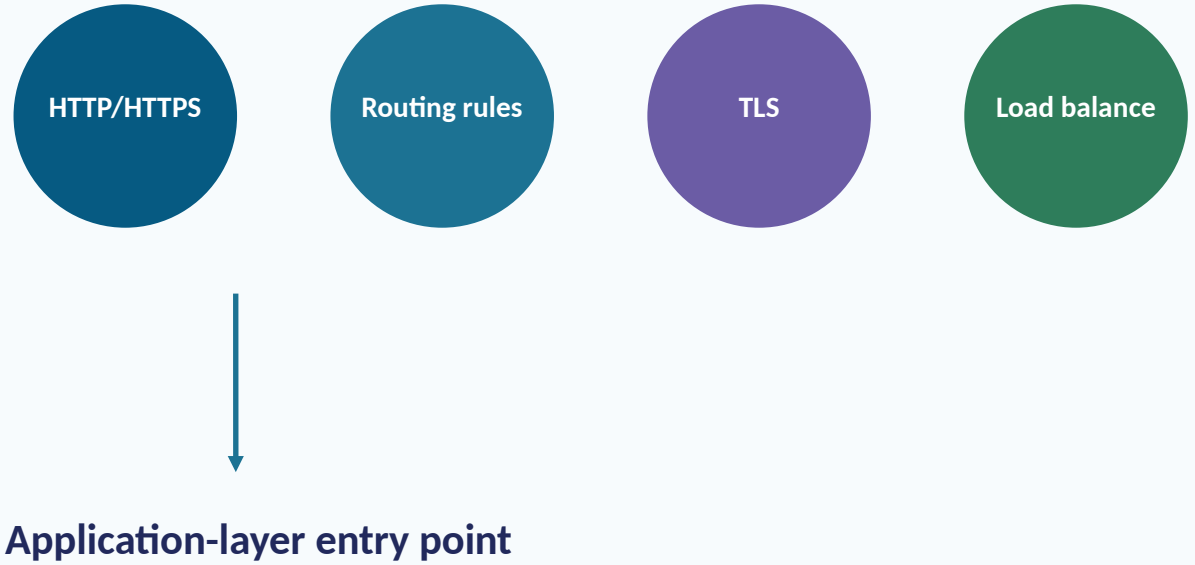
What is Ingress?

A

An API object that maps HTTP/HTTPS requests to Services inside the cluster.

Ingress

- HTTP / HTTPS routing
- Path-based and host-based routing
- TLS termination
- Load balancing
- One external entry point for multiple Services



How Ingress works — request flow

A

The controller turns the declarative Ingress rules into actual routing behavior.



Mental model

Ingress does not replace the Service. It decides which Service should receive the request; the Service still selects Pods.

Source flow: User → Ingress Controller → Ingress Rules → Service → Pods.

Ingress lives at Layer 7

A

That is why it understands HTTP concepts that NodePort and LoadBalancer do not.

- **7 Application**
- 6 Presentation
- 5 Session
- 4 Transport
- 3 Network
- 2 Data Link
- 1 Physical

What Layer 7 unlocks

HOST routing
example.com → app1

PATH routing
/api → api-service

TLS termination
HTTPS → backend Service

HTTP-aware load balancing

Layer 4 choices can expose a port. Ingress can interpret the request itself.

Ingress Resource ≠ Ingress Controller

A

One declares the rules; the other implements them.

Ingress Resource

Defines routing rules in the Kubernetes API.

Examples:
- host + path
- backend Service
- TLS configuration

Ingress Controller

Watches Ingress objects and configures the data plane.

Kubernetes does NOT ship a built-in controller.

You install one appropriate to your environment.

Operational rule: an Ingress object with no functioning controller is just a routing declaration — traffic will not magically appear.

Local development: an Ingress controller on kind

A

Convenience for a lab cluster — not a production architecture recommendation.

$ Terminal 2 — leave it running, then terminal 1

```
cloud-provider-kind

# terminal 1
kubectl get ingressclass
kubectl get gatewayclass
```

Output

```
NAME                                CONTROLLER
cloud-provider-kind (default)       kind.sigs.k8s.io/
                                    ingress-controller

NAME                    ACCEPTED
cloud-provider-kind     True
```

Why cloud-provider-kind

minikube has an ingress addon; kind has cloud-provider-kind (Module 4) — an Ingress AND Gateway API controller. ingress-nginx, the old default, was retired on March 24, 2026.

Ingress manifest — path-based routing

A

Full `networking.k8s.io/v1` object; the controller must be installed separately.

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-routing
spec:
  ingressClassName: cloud-provider-kind   # which controller should handle this
  rules:
    - host: example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
```

On kind the class is cloud-provider-kind; on a cloud it is the controller you install. The backend Services must already exist.

Apply, inspect, test

A

Pair every command with what a learner should expect to see.

$ Terminal

```
kubectl apply -f ingress.yaml
kubectl get ingress web-routing
IP=$(kubectl get ingress web-routing \
  -o jsonpath='{..ip}')
curl -H "Host: example.com" http://$IP/api
curl -H "Host: other.com" http://$IP/
```

Output

```
NAME          CLASS                  HOSTS         ADDRESS
web-routing   cloud-provider-kind    example.com   172.19.0.5

api | pod=api-5565f8b48-js24r | path=/api

404   ← other host: no rule matched
```

If ADDRESS stays empty, check the controller first: kubectl get ingressclass. An Ingress object alone cannot serve traffic.

Path-based vs host-based routing

A

Both are HTTP routing decisions; use the one that matches the URL contract you want.

Path-based

```
rules:
  - host: example.com
    http:
      paths:
        - path: /api
          pathType: Prefix
          backend:
            service:
              name: api-service
              port:
                number: 80
```

Host-based

```
rules:
  - host: api.example.com
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: api-service
              port:
                number: 80
```

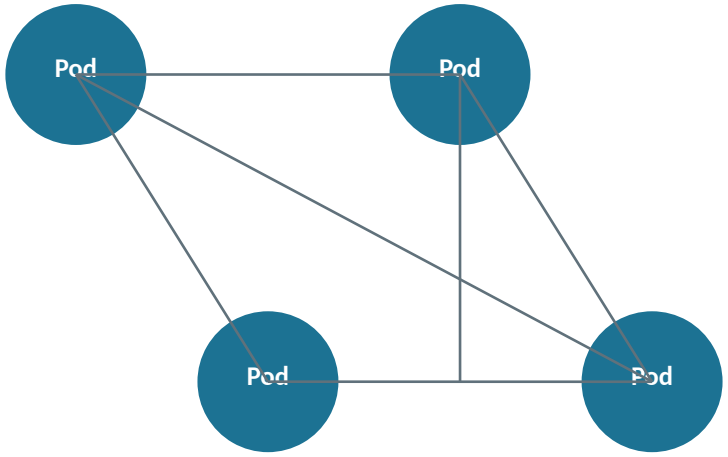
Lab: web.local → web and api.local → api on the same IP (demo.html Step 4). One Ingress can combine host and path rules.

Why NetworkPolicy?

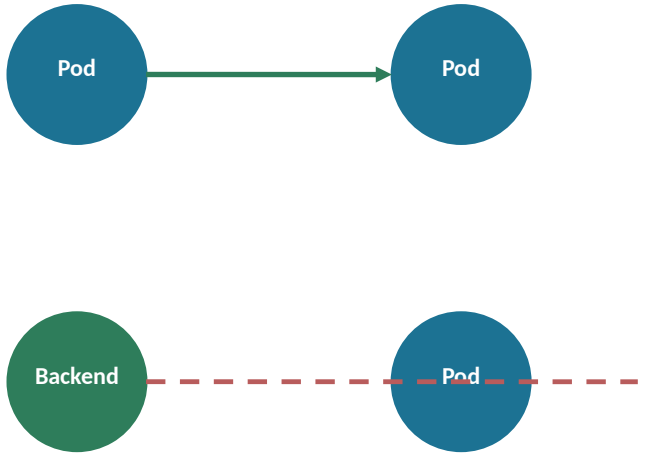
B

A default-open Pod network is easy to use — and easy to over-trust.

Before ● open Pod mesh



After ● explicit allow-list



NetworkPolicy selects which traffic is allowed to/from Pods. Enforcement requires a network plugin that supports NetworkPolicy.

What is a NetworkPolicy?

B

A declarative allow-list for inbound and outbound traffic.

Pod selector
Which Pods are protected

→

Ingress
Incoming traffic

→

Egress
Outgoing traffic

→

Selectors
Pods / namespaces / IPs

Pod selector
Targets the Pods the policy applies to.

Namespace selector
Allow traffic from selected namespaces.

ipBlock
Allow or deny traffic by IP range.

NetworkPolicy manifest — allow frontend → backend

B

The policy targets backend Pods and permits TCP/80 only from frontend Pods.

networkpolicy.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 80
```

Important: without an Egress policy, backend Pods are still free to initiate outbound traffic.

Test the policy: allowed vs blocked

B

The contrast is the lesson: matching source labels succeed; other Pods are denied.

$ Terminal

```
kubectl exec -n np-lab frontend -- \
  curl -s -m 3 -o /dev/null \
  -w '%{http_code}\n' backend-service
kubectl exec -n np-lab test-pod -- \
  curl -s -m 3 -o /dev/null \
  -w '%{http_code}\n' backend-service
```

Output

```
ALLOWED
200

BLOCKED
000
command terminated with exit code 28
```

Read the policy backwards

Target = backend. Source = frontend. Port = TCP/80. If any of those do not match, the connection is not allowed by this policy.

Egress rules — control outbound traffic

B

Ingress protects what enters a Pod; Egress protects what leaves it. Here: the frontend may only call the backend.

networkpolicy-egress.yaml

```
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 80
```

Result

Frontend Pods may open connections only to backend Pods on TCP/80.

Everything else is blocked — including DNS. Allow UDP/TCP 53 to kube-dns, or names stop resolving.

Lab: default-deny, then allow DNS, then this rule (demo.html Step 8). A connection needs egress at the source AND ingress at the destination.

NetworkPolicy prerequisite

B

A YAML policy is not enforcement by itself.

1 • Policy object

Kubernetes stores the desired traffic rules.

2 • Network plugin

Your CNI must implement NetworkPolicy enforcement. On kind, kindnet does it with its built-in kube-network-policies.

3 • Data plane

The plugin programs the actual traffic controls.

“Policy applied but traffic still flows” → check the plugin first. On kind: kubectl -n kube-system logs ds/kindnet | grep kube-network-policies

Why Helm?

Raw YAML is explicit. Helm adds packaging, reuse, configuration and release history.

Raw manifests

deployment.yaml
service.yaml
ingress.yaml
config.yaml
…

Repeat, copy, edit, drift.

Helm

Chart = package
Values = configuration inputs
Release = deployed instance

One workflow can install, upgrade, inspect history, and roll back.

Analogy from the runbook: Helm is a package manager for Kubernetes — similar in spirit to apt/yum/homebrew for an OS.

Helm architecture — from chart to release

A reusable chart becomes concrete Kubernetes resources through templating.



The key distinction

A Chart is the package. A Release is one installed instance of that package with a revision history.

Three Helm concepts to keep straight

Chart, Values and Release are the vocabulary you will use constantly.

Chart

Packaged application template.

Contains Chart.yaml, values.yaml and templates/.

Values

Configuration inputs.

Defaults live in values.yaml; users can override them at install/upgrade time.

Release

A deployed instance of a Chart.

Has a name, status and revision history.

Install your first Helm chart

We use podinfo — a small, maintained demo chart. (Bitnami's free chart catalogue changed in 2025.)

$ Terminal

```
helm repo add podinfo \
  https://stefanprodan.github.io/podinfo
helm repo update
helm install storefront podinfo/podinfo \
  --version 6.15.0 -n helm-lab \
  --create-namespace --wait
helm list -n helm-lab
```

Output

```
NAME: storefront
NAMESPACE: helm-lab
STATUS: deployed
REVISION: 1

NAME        REVISION  STATUS    CHART
storefront  1         deployed  podinfo-6.15.0
```

--version pins the chart. Without it you get whatever is newest today — and something else next month. Pin versions in anything repeatable.

Customize with values.yaml — 3 steps

Separate “what the chart can do” from “what this environment wants.”

Step 1 • extract defaults

```
helm show values \
  podinfo/podinfo

# copy only what you
# change into
# my-values.yaml
```

Step 2 • edit

```
replicaCount: 2
ui:
  message: "Hello from
    Helm values"
resources:
  limits:
    memory: 64Mi
```

Step 3 • upgrade

```
helm upgrade storefront \
  podinfo/podinfo \
  --version 6.15.0 \
  -n helm-lab \
  -f my-values.yaml
```

Output

```
STATUS: deployed     REVISION: 2
storefront-podinfo   2/2     "message": "Hello from Helm values"
```

Helm history & rollback

Same mental model as Deployment rollout history: a change creates a new revision.

$ Terminal

```
helm history storefront -n helm-lab

helm rollback storefront 2 -n helm-lab
```

Output

```
REV  STATUS      DESCRIPTION
1    superseded  Install complete
2    superseded  Upgrade complete
3    superseded  Upgrade complete
4    deployed    Rollback to 2

Rollback was a success! Happy Helming!
```

Important: rollback creates a NEW revision. It does not rewind the revision counter.

Uninstall a Helm release

Release removal is one command — then verify the resources are gone.

$ Terminal

```
helm uninstall storefront \
  -n helm-lab --keep-history

helm list -n helm-lab --uninstalled
```

Output

```
release "storefront" uninstalled

NAME        REVISION  STATUS
storefront  5         uninstalled
```

Operational note

--keep-history removes the app but keeps the release records, so helm rollback can bring it back. Without it, Helm deletes its history too.

Create your own Helm Chart

Start with the scaffold, then replace the generated templates with your application's contract.

$ Terminal

```
helm create mychart
helm lint mychart
helm template myapp \
  mychart
```

Generated structure

```
mychart/
├── Chart.yaml     name, version, appVersion
├── values.yaml    default settings
├── charts/        dependencies
├── templates/     deployment, service, ingress,
│                  httproute, hpa, tests/ …
└── .helmignore
```

Trap: image tag "" falls back to appVersion 1.16.0 — nginx from 2019. Always set the tag, and check with helm lint and helm template.

Templating a Deployment

Static YAML becomes parameterized YAML through Helm template expressions.

templates/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
        - name: myapp
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - containerPort: 80
```

{{ .Release.Name }} = release context • {{ .Values.x }} = values supplied by the chart/user

values.yaml + deploy the custom chart

The template reads configuration; the values file supplies it.

values.yaml

```
replicaCount: 2
image:
  repository: nginx
  tag: "1.29-alpine"
```

$ Terminal

```
helm install myapp ./mychart
kubectl get deployments
```

Output

```
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
myapp    2/2     2            2           18s
```

Helm command reference — keep this handy

C

The daily workflow fits into a small set of verbs.

helm repo add / update

Manage chart sources

helm search repo

Find charts in configured repos

helm install

Create a release

helm upgrade

Change a release

helm rollback

Return to a prior revision

helm uninstall

Remove a release

helm list

See installed releases

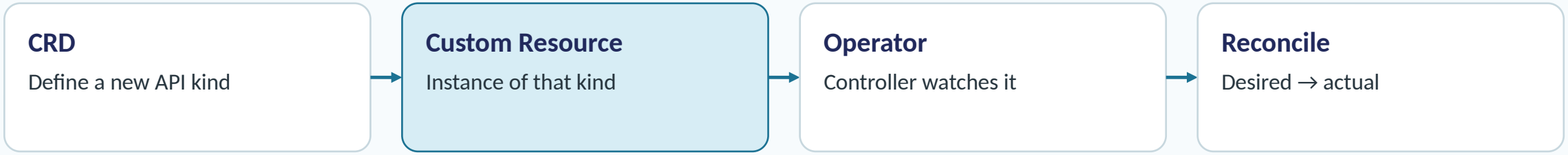
helm history

Inspect release revisions

Beyond built-in resources: CRDs & Operators

D

Gateway API itself is a set of CRDs — after the lab, try: kubectl get crd | grep gateway



Connection to Module 2

Operators extend the same controller/reconciliation pattern introduced with controller-manager — but for domain-specific resources and workflows.

Capstone lab — expose, restrict, package

E

One workflow that combines the final module's three skills.

1. **Deploy two Services**
   web + api
2. **Route with Ingress**
   /api and / on example.com
3. **Lock down with NetworkPolicy**
   default deny + allow-list
4. **Package with Helm**
   podinfo 6.15.0
5. **Publish via Gateway API**
   HTTPRoute from values

$ Terminal • capstone checkpoints

```
kubectl get ingress
kubectl get networkpolicy -n np-lab
helm history storefront -n helm-lab
kubectl get gateway,httproute -A
```

Goal: reason from traffic flow → policy boundary → packaged deployment, not memorize isolated commands.

Networking in 2026: Gateway API is the direction

F

The Ingress resource remains valid and stable, but the Kubernetes project recommends Gateway for new designs.

Ingress

Stable API • frozen

Simple model:
Ingress resource + controller

Good to understand because existing clusters still use it.

⚠️ Ingress-NGINX retired March 24, 2026; no further releases or security fixes.

Gateway API

Recommended long-term direction

GatewayClass → Gateway → HTTPRoute

More expressive routing and stronger role separation.

Gateway API v1.5 (Feb 2026) is the modern reference point.

cloud-provider-kind implements both — it even turns each Ingress into a Gateway + HTTPRoute (demo.html Step 3).

What changed by 2026 — and what this module uses

F

Every command in the lab was verified on Kubernetes v1.37, kind v0.33, cloud-provider-kind v0.11 and Helm v4.3.

Retired or changed

ingress-nginx — retired March 24, 2026
→ cloud-provider-kind here; a maintained controller or Gateway API in production

Bitnami free charts — catalogue moved (2025)
→ podinfo, pinned to 6.15.0

minikube ingress addon — minikube only
→ kind + cloud-provider-kind

New and current

Helm 4 (Nov 2025): server-side apply by default, --wait watcher, --rollback-on-failure

Gateway API v1.5 (Feb 2026): HTTPRoute with weights and headers, ListenerSet, TLSRoute

kind's kindnet enforces NetworkPolicy

helm create now scaffolds an HTTPRoute

The concepts on slides 3–30 are unchanged; only the tools around them moved on.

Lab: one Ingress — path and host routing

A

Two apps, one IP. The Host header and the path pick the Service.

$ Terminal

```
kubectl apply -f ingress.yaml -f ingress-hosts.yaml
kcurl -H "Host: example.com" http://$IP/api/users
kcurl -H "Host: example.com" http://$IP/apix
kcurl -H "Host: api.local" http://$IP/hello
kcurl -H "Host: other.com" http://$IP/
kubectl get gateway,httproute
```

Output

```
api | pod=api-5565f8b48-js24r | path=/api/users
web | pod=web-5f6b677b-hngdt | path=/apix
api | pod=api-5565f8b48-js24r | path=/hello
404

gateway/kind-ingress-gateway  172.19.0.5  True
httproute/web-routing-a379a6f6ee  ["example.com"]
```

Takeaway

Prefix matches whole path segments (/apix → web). cloud-provider-kind turned the Ingress into a Gateway + HTTPRoute. Steps 2–4

Lab: TLS termination at the Ingress

A

The Ingress holds the certificate; the Pods never see the private key.

$ Terminal

```
openssl req -x509 -nodes -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=example.com" …
kubectl create secret tls example-tls \
  --cert=tls.crt --key=tls.key
kubectl patch ingress web-routing --type=merge -p \
  '{"spec":{"tls":[{"hosts":["example.com"],
     "secretName":"example-tls"}]}}'
```

Output

```
NAME          HOSTS         ADDRESS      PORTS
web-routing   example.com   172.19.0.5   80, 443

curl --cacert tls.crt https://example.com/api
api | pod=api-5565f8b48-js24r | path=/api

without --cacert:  exit code 60 (not trusted)
```

Takeaway

A TLS Secret + a tls: block = HTTPS. In production, cert-manager and Let's Encrypt issue and renew the certificate. Step 5

Lab: only the frontend may call the backend

B

Open by default → an allow-list, enforced by kindnet.

$ Terminal

```
kubectl apply -f allow-frontend-to-backend.yaml
for p in frontend test-pod; do
  kubectl exec -n np-lab $p -- curl -s -m 3 \
    -o /dev/null -w "$p -> %{http_code}\n" \
    backend-service
done
kubectl label pod test-pod -n np-lab \
  app=frontend --overwrite   # test again
```

Output

```
before:  frontend -> 200   test-pod -> 200

after:
frontend -> 200
test-pod -> 000   (exit code 28: dropped)

test-pod relabelled   -> 200
other/frontend        -> 000   (other namespace)
```

Takeaway

Labels are identity: whoever can set labels can pass the policy. A podSelector alone means this namespace only. Steps 6–7

Lab: default deny and the DNS trap

B

Deny everything, then open only the paths the app needs.

$ Terminal

```
kubectl apply -f default-deny.yaml
kubectl exec -n np-lab frontend -- \
  nslookup -timeout=2 backend-service

kubectl apply -f allow-dns.yaml
kubectl apply -f frontend-egress.yaml
```

Output

```
;; connection timed out; no servers could be reached

after allow-dns:  name resolves, call still blocked

after frontend-egress:
frontend -> backend        200
frontend -> web.default    000
test-pod -> backend        000
```

Takeaway

Always allow DNS to kube-dns. A connection needs egress at the source AND ingress at the destination. Step 8

Lab: values, the --set trap and rollback

Every upgrade starts from the chart defaults plus what you pass this time.

$ Terminal

```
helm upgrade storefront podinfo/podinfo \
  --version 6.15.0 -n helm-lab \
  -f my-values.yaml             # rev 2
helm upgrade storefront podinfo/podinfo \
  --version 6.15.0 -n helm-lab \
  --set replicaCount=3          # rev 3
helm rollback storefront 2 -n helm-lab  # rev 4
```

Output

```
rev 2  2 replicas  "Hello from Helm values"
rev 3  3 replicas  "greetings from podinfo v6.15.0"
       ↑ message, colour and limits are gone

REVISION  STATUS      DESCRIPTION
3         superseded  Upgrade complete
4         deployed    Rollback to 2
```

Takeaway

Pass the same -f values files on every upgrade — that's what CI/CD does. A rollback is a new revision. Steps 10–12

Lab: your own chart — create, lint, template, test

The scaffold is production-shaped; its defaults still need a human.

$ Terminal

```
helm create mychart
# values.yaml: replicaCount: 2
#              tag: "1.29-alpine"
helm lint mychart
helm template myapp mychart \
  | grep -E '^kind:|image:'
helm install myapp ./mychart -n helm-lab --wait
helm test myapp -n helm-lab
```

Output

```
1 chart(s) linted, 0 chart(s) failed
kind: ServiceAccount
kind: Service
kind: Deployment
          image: "nginx:1.29-alpine"

myapp-mychart   2/2     2            2
Phase:          Succeeded
```

Takeaway

An empty tag means appVersion — nginx 1.16.0 from 2019. helm template shows the YAML without touching the cluster. Step 13

Lab: Gateway API — split traffic, guard the door

D

The platform team owns the Gateway; app teams attach HTTPRoutes.

$ Terminal

```
kubectl apply -f gateway.yaml -f shop-route.yaml
for i in $(seq 1 100); do
  kcurl -H "Host: shop.local" http://$GW/
done | cut -d' ' -f1 | sort | uniq -c
kubectl apply -f sneaky-route.yaml   # team-b
helm upgrade storefront … -f my-values.yaml \
  -f gateway-values.yaml
kubectl label ns helm-lab shop-gateway-access=true
```

Output

```
shop-gateway  cloud-provider-kind  172.19.0.6  True

     16 api
     84 web

sneaky-route:  Accepted=False (NotAllowedByListeners)

podinfo.local → "Hello from Helm values"
```

Takeaway

Weights and header matches are standard. allowedRoutes decides who may attach — nobody hijacks a hostname with YAML. Steps 14–15

Knowledge check

Nine production scenarios — Ingress, NetworkPolicy, Helm and Gateway API.

Choose one answer each, then check the key on the last quiz slide.

MODULE 10 • QUIZ

Question 1 of 9 — Ingress with no address

Q

Read the scenario, then choose one answer.

$ Scenario

```
kubectl apply -f ingress.yaml
kubectl get ingress shop
```

Output

```
NAME   CLASS   HOSTS          ADDRESS   PORTS   AGE
shop   nginx   shop.example             80      15m
```

The Ingress is 15 minutes old, has no ADDRESS, and nothing answers. What do you check first?

A. Restart the backend Pods
B. Whether a controller for class nginx is running (kubectl get ingressclass)
C. Change pathType to Exact
D. Add a NodePort Service for every backend

Question 2 of 9 — Path matching

Q

Read the scenario, then choose one answer.

$ Scenario

```
# rules: /api (Prefix) → api-service
#        /    (Prefix) → web-service
curl -H "Host: example.com" $IP/apix
```

Output

```
web | pod=web-5f6b677b-hngdt | path=/apix
```

A developer expected /apix to reach the api app. Why did it go to web?

A. Prefix matches whole path segments — /apix is not under /api
B. The api Pods were not ready
C. Ingress paths are case-insensitive
D. Prefix only matches the exact path /api

Question 3 of 9 — TLS warning

Q

Read the scenario, then choose one answer.

$ Scenario

```
kubectl get ingress shop
curl https://shop.example/
```

Output

```
NAME   HOSTS          PORTS
shop   shop.example   80, 443
curl: (60) SSL certificate problem:
  self-signed certificate
```

Customers see a certificate warning. The Ingress uses Secret shop-tls with a self-signed certificate. Best fix?

A. Move TLS into every Pod
B. Switch the Service to NodePort
C. Put a certificate from a trusted CA in shop-tls — e.g. cert-manager + Let's Encrypt
D. Delete the tls: block

Question 4 of 9 — Policy not enforced

Q

Read the scenario, then choose one answer.

$ Scenario

```
kubectl apply -f deny-db.yaml
kubectl exec test-pod -- \
  curl -s -o /dev/null -w '%{http_code}' db-api
```

Output

```
networkpolicy…/deny-db created
200
```

The YAML is correct and applied, yet every Pod can still reach the database API. Most likely cause?

A. The network plugin doesn't enforce NetworkPolicy
B. Pods must be restarted to pick up policies
C. NetworkPolicy only protects Services, not Pods
D. kube-proxy must run in IPVS mode

Question 5 of 9 — Default deny

Q

Read the scenario, then choose one answer.

$ Scenario

```
kubectl apply -f default-deny.yaml
kubectl apply -f app-to-db.yaml   # TCP 5432
kubectl logs deploy/app
```

Output

```
error: could not translate host name
  "db-service" to address
```

After a default-deny and an egress rule to the database on 5432, the app can't find the database. Fix?

A. Delete the default-deny policy
B. Use hostNetwork: true
C. Put the database IP in /etc/hosts
D. Also allow egress to kube-dns on UDP/TCP 53

Question 6 of 9 — Selector AND vs OR

Q

Read the scenario, then choose one answer.

$ Scenario

```
ingress:
  - from:
      - namespaceSelector:
          matchLabels: {team: monitoring}
      - podSelector:
          matchLabels: {app: prometheus}
```

Output

```
# two list items under from:
```

The goal: only Prometheus Pods in the monitoring namespace. What does this rule actually allow?

A. Only prometheus Pods in monitoring namespaces
B. Any Pod in a monitoring namespace, OR prometheus Pods in the policy's own namespace
C. Nothing — the YAML is invalid
D. Every Pod in every namespace

Question 7 of 9 — Helm upgrade lost settings

Q

Read the scenario, then choose one answer.

$ Scenario

```
helm upgrade web ./chart -f prod-values.yaml   # rev 7
helm upgrade web ./chart --set image.tag=1.4.2  # rev 8
kubectl get deploy web
```

Output

```
NAME   READY   UP-TO-DATE   AVAILABLE
web    1/1     1            1
# was 6/6, and the memory limits are gone
```

A hotfix with --set dropped to 1 replica and lost its limits. Why, and what's the habit?

A. Helm bug — reinstall the release
B. Use kubectl edit instead of Helm
C. Upgrade starts from chart defaults + what you pass: pass the same -f files every time
D. Add --force to every upgrade

Question 8 of 9 — Rollback

Q

Read the scenario, then choose one answer.

$ Scenario

```
helm history web
```

Output

```
REVISION  STATUS      DESCRIPTION
4         superseded  Upgrade complete
5         deployed    Upgrade complete
# revision 5 broke checkout
```

Revision 4 was good. What do you run, and what does history show afterwards?

A. helm rollback web 4 → a new revision 6, "Rollback to 4"
B. helm rollback web 4 → history is cut back to revision 4
C. helm uninstall web, then install again
D. kubectl rollout undo deploy/web — Helm follows

Question 9 of 9 — Gateway attach refused

Q

Read the scenario, then choose one answer.

$ Scenario

```
kubectl get httproute shop -n team-b \
  -o jsonpath='{..conditions[0].reason}'
```

Output

```
NotAllowedByListeners
```

team-b's HTTPRoute for shop.local was refused by the shared Gateway. Why, and who fixes it?

A. team-b's Service is missing — team-b adds it
B. HTTPRoute needs an ingressClassName
C. Gateway API can't cross namespaces
D. allowedRoutes doesn't include team-b — the platform team decides (e.g. labels the namespace)

Answer key

Every answer names the slide and the demo.html step that teach it.

Answers 1–5

```
Q1  B  check the controller
    → slides 7–8 • Step 1
Q2  A  Prefix = whole segments
    → slides 10, 33 • Step 3
Q3  C  trusted certificate
    → slide 34 • Step 5
Q4  A  CNI must enforce
    → slide 17 • Step 6
Q5  D  allow DNS
    → slide 36 • Step 8
```

Answers 6–9

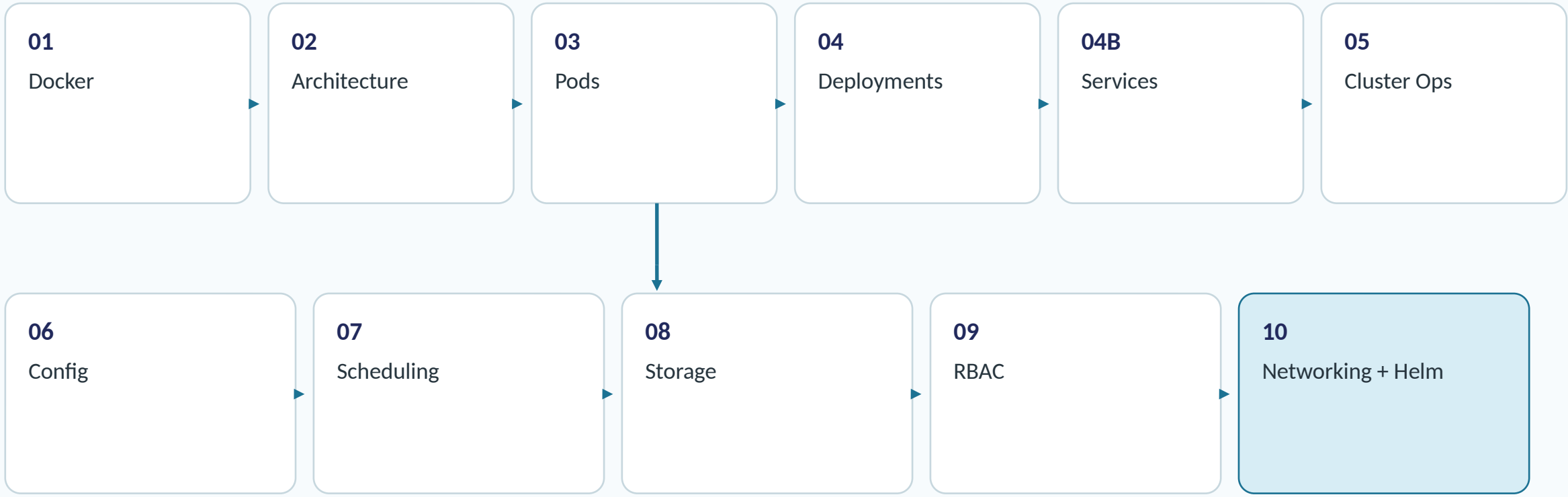
```
Q6  B  two items = OR
    → slides 13, 36 • Step 8
Q7  C  same -f files every time
    → slide 37 • Step 11
Q8  A  rollback = new revision
    → slides 23, 37 • Step 12
Q9  D  allowedRoutes
    → slide 39 • Step 15
```

Score: 8–9 course complete • 6–7 re-read what you missed • 5 or fewer: redo the matching demo.html steps.

The whole course, in one picture

F

Ten modules. One operational mental model.



Capstone mindset

Build → expose → observe → constrain → package → evolve

Where to go next

F

The course ends here; the learning path does not.

Kubernetes docs

Concepts + task guides

Build the habit:
read → deploy → break → troubleshoot → repeat.

CNCF landscape

Discover the ecosystem

Build the habit:
read → deploy → break → troubleshoot → repeat.

kind / minikube

Repeat the labs locally

Build the habit:
read → deploy → break → troubleshoot → repeat.

Community

Slack, forums, issue trackers

Build the habit:
read → deploy → break → troubleshoot → repeat.

Best next step: take the capstone lab and deliberately introduce failures — missing controller, wrong selector, blocked policy, bad values — then diagnose them.

You made it to the end.

Kubernetes is no longer a list of objects — it is a system of controllers, APIs, networking, policy, storage and packaging.

Keep building. Keep breaking. Keep learning.